



JOMO KENYATTA UNIVERSITY OF AGRICULTURE & TECHNOLOGY

SCHOOL OF OPEN, DISTANCE AND eLEARNING

P.O. Box 62000, 00200

Nairobi, Kenya

E-mail: elearning@jkuat.ac.ke

BIT 2114 Object Oriented Analysis and Design

LAST REVISION ON April 28, 2014





This presentation is intended to be covered within one week. The notes, examples and exercises should be supplemented with a good textbook. Most of the exercises have solutions/answers appearing elsewhere and accessible by clicking the green **Exercise** tag. To move back to the same page click the same tag appearing at the end of the solution/answer.

Errors and omissions in these notes are entirely the responsibility of the author who should only be contacted through the **Department of Curricula & Delivery (SODeL)** and suggested corrections may be e-mailed to elarning@jkuat.ac.ke.



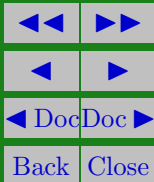
LESSON 5

UML Diagrams

Learning outcomes

Upon completing this topic, you should be able to:

1. Describe the systems behavior from the user's point of view.
2. Explain and use the concept of Use case diagrams.
3. Draw use case diagrams of the requirements captured in the previous week.





5.1. The Unified Modeling Language (UML)

UML is a language for specifying, constructing, visualizing, and documenting the artifacts of a software-intensive system.

First and foremost, the Unified Modeling Language fuses the concepts of Booch, OMT, and OOSE. The result is a single, common, and widely usable modeling language for users of these and other methods.

Second, the Unified Modeling Language pushes the envelope of what can be done with existing methods. As an example, the UML authors targeted the modeling of concurrent, distributed systems to assure the UML adequately addresses these domains.

Third, the Unified Modeling Language focuses on a standard modeling language, not a standard process. Although the UML must be applied in the context of a process, it is our ex-



BIT 2114 Object Oriented Analysis and Design

perience that different organizations and problem domains require different processes. (For example, the development process for shrink-wrapped software is an interesting one, but building shrink-wrapped software is vastly different from building.

UML is a Language for Specifying, Visualizing, Constructing, and Documenting Software Artifacts. A Modeling Language Provides the following:

1. Model Elements: Concepts and Semantics
2. Notation: Visual Rendering of Model Elements
3. Guidelines: Hints and Suggestions for Using Elements in Notation

The primary design goals of the UML are as follows:

1. Provide users with a ready-to-use, expressive visual modeling language to develop and exchange meaningful models.



BIT 2114 Object Oriented Analysis and Design

2. Furnish extensibility and specialization mechanisms to extend the core concepts.
3. Support specifications that are independent of particular programming languages and development processes.
4. Provide a formal basis for understanding the modeling language.
5. Encourage the growth of the object tools market.
6. Support higher-level development concepts such as components, collaborations, frameworks and patterns.
7. Integrate best practices.

In terms of the views of a model, the UML defines the following graphical diagrams:

- use case diagram



- class diagram
- behavior diagrams:
 - statechart diagram
 - activity diagram
 - interaction diagrams:
 - * sequence diagram
 - * collaboration diagram
- implementation diagrams:
 - component diagram
 - deployment diagram

UML Diagrams and their Descriptions

The following table describes the UML diagrams in use. In this unit, we shall consider a few of them.

◀◀	▶▶
◀	▶
◀ Doc	Doc ▶
Back	Close



Diagram	Description
Activity Diagram	Depicts high-level business processes, including data flow, or to model the logic of complex logic within a system.
Class Diagram	Shows a collection of static model elements such as classes and types, their contents, and their relationships.
Communication Diagram	Shows instances of classes, their interrelationships, and the message flow between them. Communication diagrams typically focus on the structural organization of objects that send and receive messages. Formerly called a Collaboration Diagram.
Component Diagram	Depicts the components that compose an application, system, or enterprise. The components, their interrelationships, interactions, and their public interfaces are depicted.



Figure 5.1: UML Diagrams

5.2. System's Behavior from User's Point of View: Use Case Diagrams

To model the system's behavior from the users' point of view, a use case model is used. A use case model is an abstraction of the problem domain and helps to facilitate well articulated and unambiguous understanding of the problem domain. A use case template is used to detail the use case model. Use case models involve actors and use cases. An actor is something that interacts with the system. A use case model captures the problem domain in terms of functionality provided (use cases), and the 'roles' (actors) for whom these functions are performed; the actors are represented by stick figures and the use cases by



BIT 2114 Object Oriented Analysis and Design

ovals. An actor makes use of a system in different ways. Each of these ways is known as a use-case. A use-case may involve a number of actors, just as an individual actor may make use of several use-cases.

In summary, a use case diagram shows the relationship among use cases within a system or other semantic entity and their actors.

Semantics

Use case diagrams show actors and use cases together with their relationships. The use cases represent functionality of a system or a classifier, like a subsystem or a class, as manifested to external interactors with the system or the classifier.



Notation

A use case diagram is a graph of actors, a set of use cases, possibly some interfaces, and the relationships between these elements. The relationships are associations between the actors and the use cases, generalizations between the actors, and generalizations, extends, and includes among the use cases. The use cases may optionally be enclosed by a rectangle that represents the boundary of the containing system or classifier.

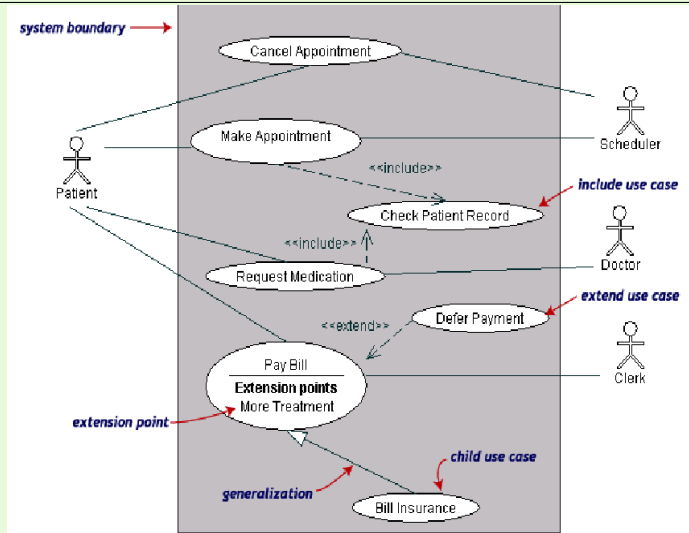


Figure 5.2: Use Case Diagram Indicating System Boundary



5.2.1. Use Cases for Requirements Capture

Use cases determine the system requirements by focusing on how the system is used. This is not really new – for a long time, people have used typical interactions with a system to help understand requirements. These have tended to be excessively informal and rarely documented.

Ivar Jacobson popularized this approach in the early 1990s in his Objectory method. Interactions with the system are captured by use cases (diagrammatically and textually). These are more structured/stylised than the previous ad hoc approaches. This is now incorporated in the Rational Unified Process (RUP).



5.2.2. Use Cases and Scenarios

A scenario is one possible interaction with a system, e.g. The customer browses the catalog and adds desired items to the shopping basket. When the customer wishes to pay, the customer describes the shipping and credit card information and confirms the sale. The system checks the authorization on the credit card and confirms the sale both immediately and with a follow-up email. (Fowler).

There could be many variations on the above:

- .e.g. the customer may change their mind about the purchases
- .e.g. the credit card authorization may fail

The set of related scenarios is called a **use case**, i.e. a use case



is a pattern of interaction with a system

5.2.3. Specifying a Use Case

A use case is often specified as a sequence of numbered steps with alternatives as variations on that sequence, e.g.

Buy a Product

1. Customer browses through catalog and selects items to buy
2. Customer goes to checkout
3. Customer fills in shipping information
4. System presents full pricing information, including shipping



BIT 2114 Object Oriented Analysis and Design

5. Customer fills in credit card information
6. System authorizes purchase
7. System confirms sale immediately
8. System sends confirming email to customer

Alternative: Authorization failure

- At step 6, system fails to authorize credit purchase
- Allow customer to re-enter credit card information and retry

Alternative: Regular customer

- 3a. System displays current shipping information, pricing information,



- and last four digits of credit card information
- 3b. Customer may accept or override these defaults
 - Return to primary scenario at step 6

5.2.4. Use Case Diagrams

Use case diagrams identify the use cases and the entities outside the system that will interact with them.

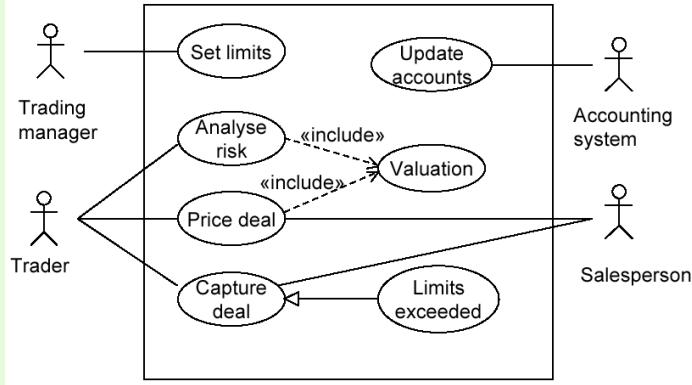


Figure 5.3: Use Case Diagram with Actors

The use case diagram has the following:

1. 4 actors (or external entities)
2. 7 use cases (or patterns of interaction)
3. 5 use cases directly accessed



- 2 use cases indirectly accessed via relations with other use cases.

The diagram is a visual summary of a number of interactions. More detailed descriptions (as indicated before) are also required.

5.2.5. What is an Actor?

An actor is a role that some external entity might play on. We are not concerned with which particular entity is involved. We are not concerned with whether the one entity may play multiple roles, i.e. may interact with the system as different actors.

- An actor doesn't have to be a person even though a stick figure is used on. e.g. accounting system.
- An actor should be a beneficiary of the system.



BIT 2114 Object Oriented Analysis and Design

- The functionality provides some benefit to the actors
- This will give a natural association which will be retained (possibly in modified form) during the development of the system
- Cockburn refers to the primary actor, the one initiating the interaction
- There are three basic approaches:
 - always show interaction with external systems
 - show interaction with external systems when they initiate contact
 - show interaction with external systems when they derive benefit



Examples

In a banking system where a customer can withdraw money, we would draw:

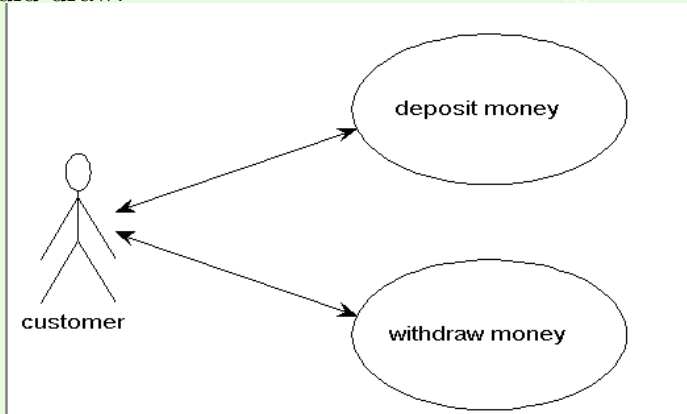


Figure 5.4: An Actor in a Banking System

Now it might be that the system which is being implemented



BIT 2114 Object Oriented Analysis and Design

in the bank needs to involve a cashier for depositing, but that to withdraw money the customer has to use the cash machine. The cashier is then an actor.

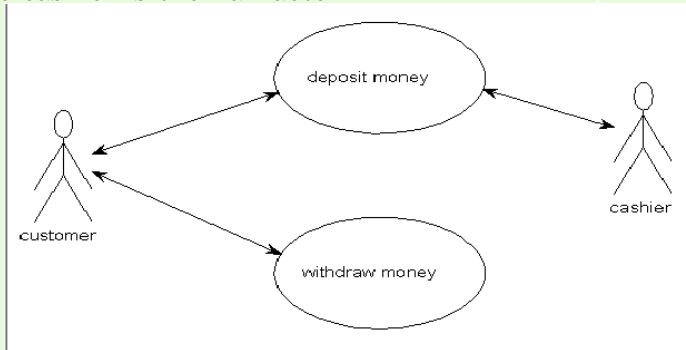


Figure 5.5: Two Actors in a Banking System

And so we build our model of the ways the system is used. We might add two more use-cases of the bank system.



JKUAT: Setting trends in higher Education, Research and Innovation

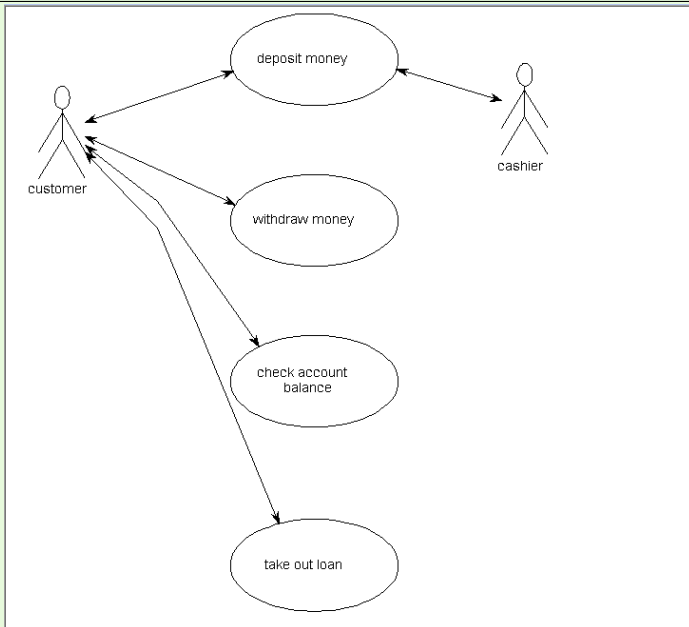




Figure 5.6: Banking System Use Cases

Examining this, we see that there is probably a need for another actor. (I never borrowed money without having to grovel to the bank manager, and the bank manager always looked carefully at my pitiful balance before grudgingly saying yes).

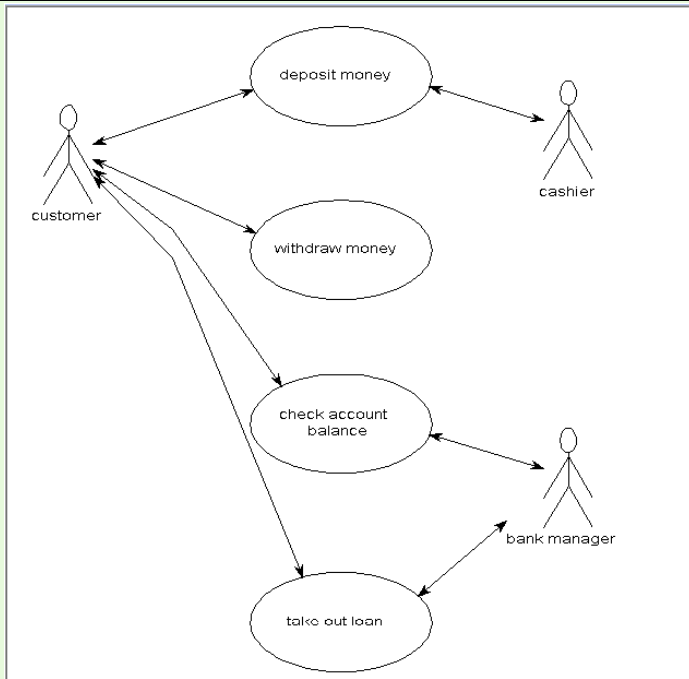




Figure 5.7: Banking System Use Cases with More Actors

Use cases may in fact use other use-cases. The withdraw cash use-case would make use of the account balance use-case before issuing the money.

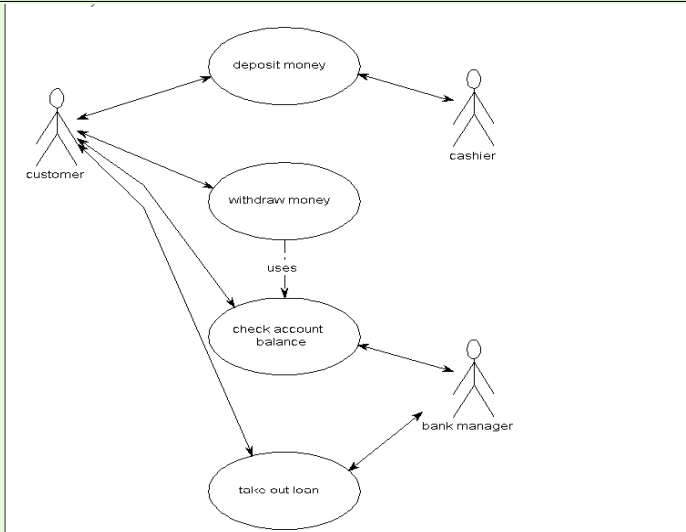


Figure 5.8: Use Cases using other use cases

A use-case is a very abstract view of what the system pro-



BIT 2114 Object Oriented Analysis and Design

vides to the various actors who use it. It is not intended to give a detailed, blow-by-blow account of how the services are provided. Therein lies the beauty of the approach. By keeping out detail, it is possible to reason at a higher level. Later the use-cases can be opened out using object-interaction diagrams to begin to define their precise operation.

5.3. Associations Between Use Cases

Similarly, there can be associations between use cases. One use case may be part of another. Shown with the ■includes■ stereotype and a dashed arrow from the base use case to the one which is included.



Use Case Extensions

- Use cases can be extended by other use cases
- Usually, conditions exist somewhere in the use case's activity that would cause the extended use case to occur
 - This 'somewhere' is called an extension point in UML
- Use cases can be extended by other use cases
 - Usually, conditions exist somewhere in the use case's activity that would cause the extended use case to occur
 - This 'somewhere' is called an extension point in UML
- The original use case is called the base use case o The new use case is called the extended use case



BIT 2114 Object Oriented Analysis and Design

- The extended use case usually augments the base use case with additional behaviour.
- For example, a use case for a hotel might be 'reserve hotel room'
 - An extension to this use case might be 'request coffee machine' to request that a coffee maker be in the room
 - The customer still wants to reserve a hotel room, but they also want a coffee maker in their room when they arrive
- Use cases extensions are specified using a dashed arrow
 - The arrow points from the extended use case to the base use case



- The arrows are labelled with **extend** plus a description of the conditions which cause the extension to occur
- The base use case usually contains a description of the extension points where the use case may be extended



Use Case Extension Example

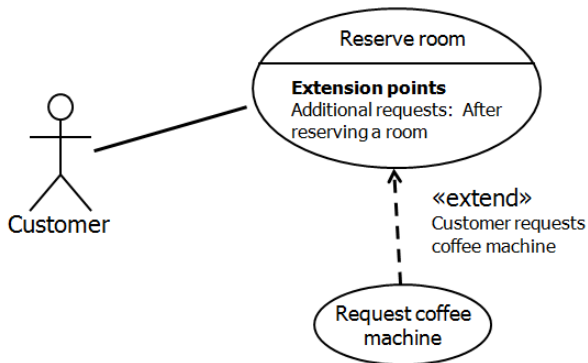


Figure 5.9: Use Cases Extension

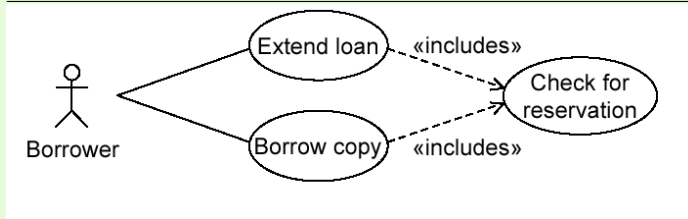


Figure 5.10: Use Cases Associations

- e.g. both analyse risk and price deal include valuation
- e.g. for a library system, both extend loan and borrow copy include check for reservation

One use case may be more general/specific than another. use the inheritance relation.



Book borrower

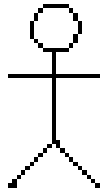




Figure 5.11: Use Cases Inheritance

- e.g. limit exceeded is a more specific use case than capture deal
- e.g. journal borrower is a more specific actor than book borrower Book borrower

Use Case Inclusions

Use case inclusion is when one use case is used within another use case

For example, the use cases 'withdraw' and 'deposit' in a bank, might be used within the use case 'transfer funds'.

Use case inclusion is specified using a dashed arrow. The arrow points from the container use case to the included use case. The arrow is labelled with **include**.



Use Case Inclusion Example

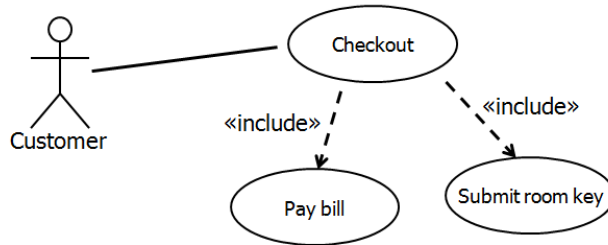


Figure 5.12: Use Cases Include

When there are significant variations of a use case

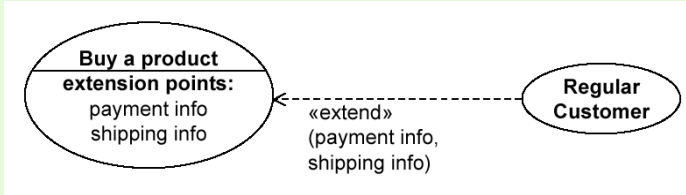


Figure 5.12: Use Cases Extends

Actor Inheritance

It may also make sense to have actor inheritance in a system, for example, an employee might have a specialization called manager. A manager might handle (in addition to normal employee duties) customer complaints, refunds, scheduling, etc.



Actor Inheritance

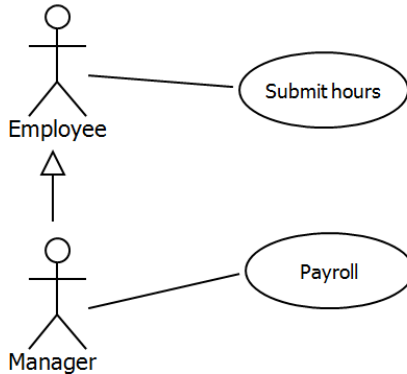


Figure 5.13: Actor Inheritance

5.4. Summary

1. Use case diagrams consist of at least two components:



- (a) An actor: represented as stick people, with a label below, naming the actor's role. There may be multiple actors in a diagram
 - (b) A use case: Use cases are represented as ellipses, with a label inside, naming the use case. There may be multiple use cases in a diagram.
2. For an example, consider an auction website (e.g. eBay)
- (a) The actor customer might:
 - i. Register for an account
 - ii. Search for items by keyword
 - iii. Place a bid on an item
 - iv. Put an item up for auction
 - v. Check the status of this item
- An actor represents a role that a user might play.



3. Use cases are not a good tool for finding classes because:
- (a) Use cases emphasize ordering ("When a customer places an order over the phone, his credit card number is validated. Then the database is updated and a confirmation number is issued", etc.). This is incompatible with object technology: the method shuns early reliance on sequentiality properties, because they are so fragile and subject to change.
 - (b) Relying on a scenario means that you focus on how users see the system's operation. But the system does not exist yet. (A previous system might exist, but if it were fully satisfactory you would not be asked to change or rewrite it.)
 - (c) Use cases favour a functional approach, based on pro-



BIT 2114 Object Oriented Analysis and Design

cesses (actions). This approach is the reverse of O-O decomposition, which focuses on data abstractions; it carries a serious risk of reverting, under the heading of object-oriented development, to the most traditional forms of functional design.

Learning Activities:

1. Travel Agent Application

Initial requirements description:

1. The desired system is an application which allows travel agent to customize trips for customers
2. Enter the requirements analysis phase



BIT 2114 Object Oriented Analysis and Design

- (a) We should add anything missing from the requirements capture phase
- (b) We should clear up any ambiguities or poorly worded phrases

3. Requirements Analysis:

- (a) What features do we need?
 - i. We need the ability to create a customized trip for the customers What does this involve?
 - ii. To answer this question, we need to converse with the customers and travel agents. We can't actually do that here, so let's just make some decisions based on what we think would be appropriate:
 - iii. Obviously, in the end, designers and developers



would never be the people to make these decisions

- iv. Assume that the following decisions were achieved through an interaction between customers, travel agents, and architects Customizing a Trip
- A. Customers will want to o Select the flights that suit them
 - B. Based on price, class, times, etc.
 - C. Choose how many days to stay
 - D. Choose their hotel & room type
 - E. Based on price, class, amenities, etc.
 - F. Add any extras to the trip
 - G. Excursions to an attraction, rental of a car, a city tour, etc.



(b) Use Cases

- i. So what are our use cases?
 - A. Customers booking a trip is one
 - B. Cancelling trips might be another
 - C. Let's keep it simple, and stick to these 2 for now
- ii. So who are our actors?
 - A. Customers
 - B. Travel Agents
- iii. Which actors participate in each use case?
 - A. Customers and Agents both participate in both use cases

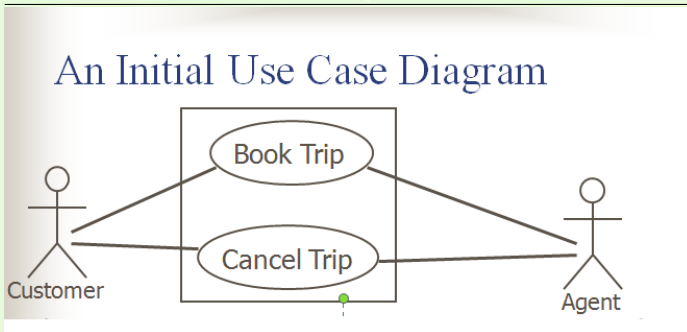


Figure 5.14: Initial Use Case Diagram

Requirements analysis occurs in a few stages:

- Identify and clarify use cases
 - Create an initial use case diagram
- Identify any use case special cases
 - Update the use case diagram to include use case extensions to accommodate these special cases





- Elaborate on use cases
 - Update the use case diagram to include any behavioural inclusion (i.e. behaviour components).
 - This typically involves the use of use case inclusion.

Additional Information

- Booking a trip should be the same for all customers, with one exception:
 - If the customer wants to book some trip extras, there will be some extra work to be done by our program
 - This could include making car rental reservations, or ordering tickets to excursions, attractions, or tours
- What UML feature should be use to represent this optional added behaviour?



Adding a Use Case Extension

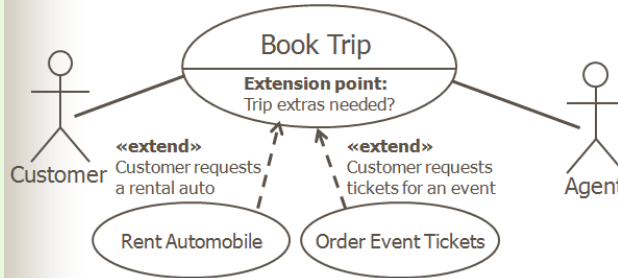


Figure 5.15: Initial Use Case Diagram with Extensions



Use Case Elaboration

- Book Trip
 - What is involved with this use case?
- Most trips include a flight and a hotel booking
 - So let's say that the Book Trip use case uses the following use cases:
 - * Book Flight.
 - * Book Hotel
- Cancel Trip
 - What is involved with this use case?
- We would need to cancel the flight, the hotel reservation, and the trip extras (if any)



- Trip extras should be considered an extension, since it only occurs in some trips
- Let's say the 'Cancel Trip' use case includes the following use cases:
 - * Cancel Flight
 - * Cancel Hotel Reservation

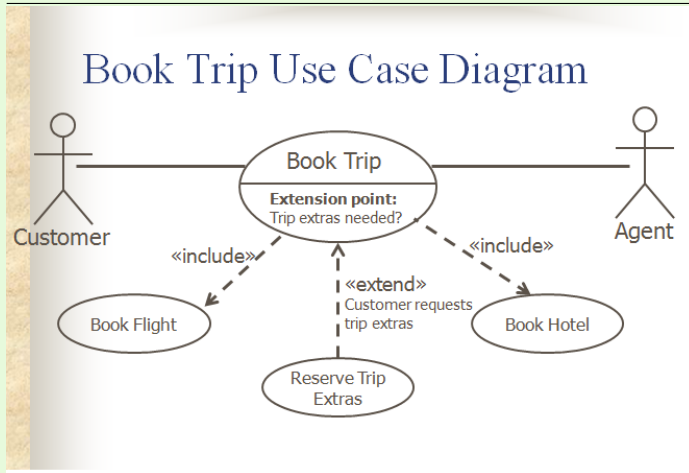


Figure 5.15: Book Trip Use Case

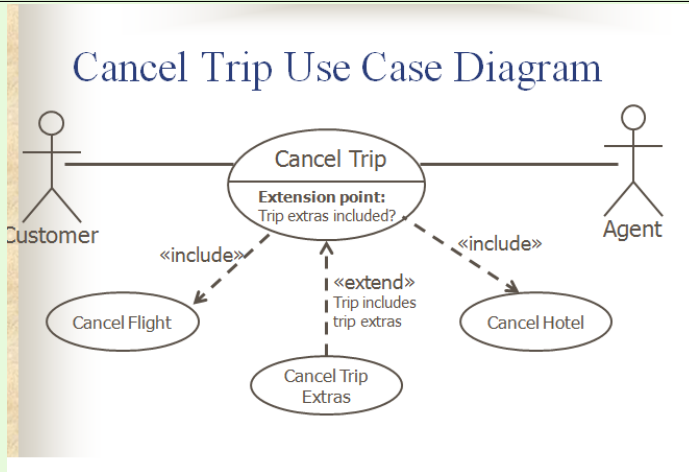


Figure 5.16: Cancel Trip Use Case

1. Draw a Use Case Diagram for the Class TimeTable System.



Revision Questions

Example  Explain the purpose of use case diagrams in object-oriented software development

Solution: for revision



EXERCISE 1.  In what ways might you expect end-users, clients and developers to disagree about problems in Information Systems Development?

EXERCISE 2.  Briefly explain FIVE steps involved in moving from the model you have developed to a detailed design of a software system supporting the Use Case



References and Additional Reading Materials

1. John. W Satzinger (2004). Object Oriented Analysis and Design with the Unified Process. Cengage Learning, ISBN-13: 978-0619216436
2. Acta Informatica ISSN 0001-5903
3. Advances in Computational Mathematics ISSN 1019-7168